C++ Chapter 00 Notes

Chapter 00 — Notes

A Few Common C++ Problems

In this section, we'll address some of the common issues that new programmers seem to run across with fairly high probability. This is not meant to be a comprehensive list of compilation or execution problems, but rather a pragmatic list of solutions to very basic issues. If you have any suggestions for other issues that might be added to this list, post them in the comments section below.

General run-time issues

Q: When executing a program, the console window blinks and then closes immediately.

First, add or ensure the following lines are near the top of your program (Visual Studio users, make sure these lines appear after `#include "pch.h"` or `#include "stdafx.h"`, if those exist):

```
#include <iostream>
#include <limits>
```

Second, add the following code at the end of your `main()` function (right before the return statement):

```
std::cin.clear(); // reset any error flags
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // ignore any characters in the input b
until we find an enter character
std::cin.get(); // get one more char from the user
```

This will cause your program to wait for the user to press a key before continuing, which will give you time to examine your program's output before your operating system closes the console window.

Other solutions, such as the commonly suggested `system("pause")` solution may only work on certain operating systems and should be avoided.

Older versions of Visual Studio may not pause when the program is run in *Start With Debugging (F5)* mode. Try running in *Start Without Debugging (Ctrl-F5)* mode.

Q: I ran my program and get a window but no output.

Your virus scanner or anti-malware may be blocking execution. Try disabling it temporarily and see if that's the issue.

Q: My program compiles but it isn't working correctly. What do I do?

Debug it! There are tips on how to diagnose and debug your programs later in chapter 3.

General compile-time issues

Q: When I compile my program, I get an error about unresolved external symbol _main or _WinMain@16

This means your compiler can't find your `main()` function. All programs must include a `main()` function.

There are a few things to check: - a) Does your code include a function named `main`? - b) Is `main` spelled correctly? - c) When you compile your program, do you see the file that contains function `main()` get compiled? If not, either move the `main()` function to one that is, or add the file to your project (see lesson 2.8 – Programs with multiple code files for more information about how to do this). - d) Did you create a console project? Try creating a new console project.

Q: When I compile my program, I get an error that main is already defined, or about multiple definitions for main

A C++ program is only allowed to have a single function named `main`. Your C++ program has more than one. Examine the files in your project and delete all `main` functions except one.

Q: I'm trying to use C++11/14/17/XX functionality and it doesn't work

If your compiler is old, it may not support these more recent additions to the language. In that case, upgrade your compiler.

For modern IDEs/compilers, your compiler may be defaulting to an older language standard. We cover how to change your language

standard in lesson 0.12 – Configuring your compiler: Choosing a language standard.

Q: When I compile my program, I get an error that it cannot open the .exe for writing

This means the linker is trying to create your executable, but it can't. This can happen for quite a few reasons:

- Most commonly, the .exe is currently running. A running executable can't be replaced while it is running. Close the running .exe and recompile.
- Your antivirus or malware protection is preventing the executable from being created or replaced.
- The .exe already exists and is currently locked for some other reason. Try rebooting (to force any locks to be released), then recompile.

In Visual Studio, this is error LNK1168.

Q: When trying to use cin, cout, or endl, the compiler says cin, cout, or endl is an 'undeclared identifier'

First, make sure you have included the following line near the top of your file:

```
#include <iostream>
```

Second, make sure each use of `cin`, `cout`, and `endl` are prefixed by `"std::"`. For example:

```
std::cout << "Broccoli" << std::endl;
```

If this doesn't fix your issue, then it may be that your compiler is out of date, or the install is corrupted. Try reinstalling and/or upgrading to the latest version of your compiler.

Q: When trying to use endl to end a printed line, the compiler says end1 is an 'undeclared identifier'

Make sure you do not mistake the letter l (lower case L) in `endl` for the number 1. `endl` is all letters. Make sure your editor is using a font that makes clear the differences between the letter lower case L, upper case i, and the number 1. Also the letter capital o and the number zero can easily be confused in many non-programming fonts.

Visual Studio issues

Q: When compiling with Microsoft Visual C++, you get a C1010 fatal error, with an error message like "c:.cpp(263) :fatal error C1010: unexpected end of file while looking for precompiled header directive"

This error occurs when the Microsoft Visual C++ compiler is set to use precompiled headers but one (or more) of your C++ code files does not `#include "stdafx.h"` or `#include "pch.h"` as the first line of the code file.

Our suggested fix is to turn off precompiled headers, which we show how to do in lesson 0.7 – Compiling your first program.

If you would like to keep precompiled headers turned on, to fix this problem, simply locate the file(s) producing the error (in the above error, test.cpp is the culprit), and add the following line at the very top of the file(s):

```
#include "pch.h"
```

Older versions of Visual Studio use `"stdafx.h"` instead of `"pch.h"`, so if `pch.h` doesn't resolve the issue, try `stdafx.h`.

Note that for programs with multiple files, every C++ code file needs to start with this line.

Q: Visual Studio gives an error like: "1MSVCRTD.lib(exe_winmain.obj) : error LNK2022: unresolved external symbol _WinMain@16 referenced in function "int __cdecl invoke_main(void)" (?invoke_main@@YAHXZ)"

You've likely created a Windows graphical application rather than a console application. Recreate your project, and make sure to create it as a Windows (or Win32) *Console* project.

Q: When I compile my program, I get a warning about "Cannot find or open the PDB file"

This is a warning, not an error, so it shouldn't impact your program. However, it is annoying. To fix it, go into the Debug menu -> Options and Settings -> Symbols, and check "Microsoft Symbol Server".

Something else

Q: I have some other problem that I can't figure out. How can I get an answer quickly?

As you progress through the material, you'll undoubtedly have questions or run into unexpected problems. What to do next depends on your problem. But in general, there are a few things you can try.

First, **ask a search engine**. Find a good way to phrase your question and do a search. If you are searching an error message, paste in the exact error message using quotes (exclude any filenames or line numbers). Odds are someone has already asked the same question and there is an answer waiting for you.

Second, **ask an AI** like ChatGPT via Bing. Start your question with "In C++," to get a C++ specific answer. Some ideas for things you can ask for:

- An explanation of some concept (e.g. "In C++, what is a local variable?").
- The difference between two things (e.g. "In C++, what is the difference between a pointer and a reference?")
- A demonstration of a concept (e.g. "In C++, write a short program that adds two numbers")

Note that AIs may return inaccurate or outdated information, and the programs they write will probably not follow modern best practices.

If the above fails, **ask on a Q&A board**. There are websites designed for programming questions and answers, like Stack Overflow. Try posting your question there. Remember to be thorough about what your problem is, and include all relevant information like what OS you're on and what IDE you're using.

Compiling Your First Program

Before we can write our first program, we need to learn how to create new programs within our Integrated Development Environment (IDE). In this lesson, we'll cover how to do that, and you'll also compile and execute your first program!

Projects

To write a C++ program inside an IDE, we typically start by creating a new project (we'll show you how to do this in a bit). A **project** is a container that holds all of your source code files, images, data files, etc… that are needed to produce an executable (or library, website, etc…) that you can run or use. The project also saves various IDE, compiler, and linker settings, as well as remembering where you left off, so that when you reopen the project later, the state of the IDE can be restored to wherever you left off. When you choose to compile your program, all of the .cpp files in the project will get compiled and linked.

Each project corresponds to one program. When you're ready to create a second program, you'll either need to create a new project, or overwrite the code in an existing project (if you don't want to keep it). Project files are generally IDE specific, so a project created for one IDE will need to be recreated in a different IDE.

> **Best practice**
>
> Create a new project for each new program you write.

Console projects

When you create a new project, you'll generally be asked what type of project you want to create. All of the projects that we will create in this tutorial will be console projects. A **console project** means that we are going to create programs that can be run from the Windows, Linux, or Mac console.

Here's a screenshot of the Windows console:

By default, console applications have no graphical user interface (GUI), they print text to the console, read input from the keyboard, and are compiled into stand-alone executable files. This is perfect for learning C++, because it keeps the complexity to a minimum, and ensures things work on a wide variety of systems.

Don't worry if you've never used a console before, or don't know how to access it. We'll compile and launch our programs through our IDEs (which will invoke the console when necessary).

Workspaces / solutions

When you create a new project for your program, many IDEs will automatically add your project to a "workspace" or a "solution" (the term varies by IDE). A workspace or solution is a container that can hold one or more related projects. For example, if you were writing a game and wanted to have a separate executable for single player and multiplayer, you'd need to create two projects. It wouldn't make sense for both of these projects to be completely independent – after all, they are part of the same game. Most likely, each would be configured as a separate project within a single workspace/solution.

Although you can add multiple projects to a single solution, we generally recommend creating a new workspace or solution for each program, especially while learning. It's simpler and there's less chance of something going wrong.

Writing your first program

Traditionally, the first program programmers write in a new language is the infamous hello world program, and we aren't going to deprive you of that experience! You'll thank us later. Maybe.

Creating a project in Visual Studio 2019 (or newer)

When you run Visual Studio 2019 (or newer), you should see a dialog that looks like this:

Select *Create a new project*.

You'll then see a dialog that looks like this:

If you've already opened a prior project, you can access this dialog via the *File menu > New > Project*.

Select *Windows Desktop Wizard* and click *Next*. If you don't see this, then you probably forgot to choose to install the *Desktop development with C++* workload when you installed Visual Studio. In that case, go back to lesson 0.6 – Installing an Integrated Development Environment (IDE) and reinstall your Visual Studio as indicated (note: rather than doing a full reinstall, you can run the Visual Studio installer and modify your existing installation to add the C++ workload).

Next, you'll see a dialog that looks like this:

Replace the existing project name with `HelloWorld`.

It's recommended that you also check the *Place solution and project in the same directory*, as this reduces the number of subdirectories that get created with each project.

Click *Create* to continue.

Finally, you'll see one last dialog:

Make sure the *Application type* is set as *Console Application (.exe)* and that the *Precompiled Header* option is unselected. Then click *OK*.

You've now created a project! Jump down to the Visual Studio Solution Explorer section below to continue.

> **Q: What are precompiled headers and why are we turning them off?**
>
> In large projects (those with many code files), precompiled headers can improve compilation speed by avoiding some redundant compilation that tends to occur in larger projects.
>
> However, precompiled headers require extra work to use, and for small projects (such as those you'll create in our tutorials) make little to no difference in compilation times.
>
> For this reason, we recommend turning precompiled headers off initially, and only enabling them later if and when you find your compilation times suffering.

Creating a project in Visual Studio 2017 or older

To create a new project in Visual Studio 2017 or older, go to the *File menu > New > Project*. A dialog box will pop up that looks something like this:

First, make sure *Visual C++* is listed on the left side. If you don't see *Visual C++*, then you probably forgot to choose to install the *Desktop development with C++* workload when you installed Visual Studio. In that case, go back to lesson 0.6 – Installing an Integrated Development Environment (IDE) and reinstall your Visual Studio as indicated (note: rather doing a full reinstall, you can run the Visual Studio installer and modify your existing install to add the C++ workload).

If you're using Visual Studio 2017 v15.3 or newer, underneath *Visual C++*, select *Windows Desktop* and then select *Windows Desktop Wizard* in the main window.

If you don't see *Windows Desktop* as an option, you're probably using an older version of Visual Studio. That's fine. Instead, choose *Win32* and then *Win32 Console Application* in the main window.

Down below, in the *Name* field, enter the name of your program (replace the existing name with `HelloWorld`). In the Location field, you can optionally select a different location for your project to be placed into. The default is fine for now.

Click *OK*. If you're using an older version of Visual Studio, the Win32 Application Wizard will launch. Press *Next*.

At this point, you should see a wizard dialog that looks something like this (older versions of Visual Studio use a different style, but have most of the same options):

Make sure you uncheck *Precompiled Header*.

Then click *Ok* or *Finish*. Now your project is created!

Visual Studio Solution Explorer

On the left or right side of the window, you should see a window titled *Solution Explorer*. Inside this window, Visual Studio has created a solution for you (*Solution 'HelloWorld'*). Within that, with the name in bold, is your new project (*HelloWorld*). Within the project, Visual Studio

has created a number of files for you, including *HelloWorld.cpp* (underneath the *Source Files* tree item). You may also see some other .cpp or .h files, which you can ignore for now.

In the text editor, you will see that Visual Studio has already opened *HelloWorld.cpp* and created some code for you. Select and delete all of the code, and type/copy the following into your IDE:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

To compile your program, either press *F7* (if this doesn't work, try *Ctrl-Shift-B*) or go to the *Build menu* > *Build Solution*. If all goes well, you should see the following appear in the Output window:

```
1>------ Build started: Project: HelloWorld, Configuration: Debug Win32 ------
1>HelloWorld.cpp
1>HelloWorld.vcxproj -> c:\users\alex\documents\visual studio 2017\Projects\HelloWorld\Debug\HelloWorld.exe
========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ==========
```

This means your compile was successful!

> **Q: I got error C1010 ("fatal error C1010: unexpected end of file while looking for precompiled header. Did you forget to add '#include "stdafx.h"' to your source?"). What now?**
>
> You forgot to turn off precompiled headers when you created your project. Recreate your project (as per the instructions above) and make sure to disable precompiled headers.

To run your compiled program, press *Ctrl-F5*, or go to the *Debug menu* and choose *Start Without Debugging*. You will see something like the following:

That is the result of your program! Congratulations, you've compiled and run your first program!

> **Related content**
>
> When you run a program directly from Visual Studio, you may see an additional line of output that looks something like this:
>
> ```
> C:\Users\Alex\source\repos\Project6\Debug\Project6.exe (process 21896) exited with code 0.
> ```
>
> This is normal. Visual Studio is providing some additional information about whether your program exited normally or abnormally. We discuss this further in lesson 2.2 – Function return values (value-returning functions).

Creating a project in Code::Blocks

To create a new project, go to *File menu* > *New* > *Project*. A dialog box will pop up that looks like this:

Select *Console application* and press the *Go (or Create)* button.

If you see a console application wizard dialog, press *Next*, make sure C++ is selected and press *Next* again.

Now you will be asked to name your project. Title the project `HelloWorld`. You can save it wherever you wish. On Windows, we recommend you to save it in a subdirectory of the C drive, such as `C:\CBProjects`.

You may see another dialog asking you which configurations you want enabled. The defaults should be fine here, so select *Finish*.

Now your new project has been created.

On the left side of the screen, you should see a *Management* window, with the *Projects* tab selected. Inside that window, you'll see a *Workspace* folder, with your *HelloWorld* project inside of it:

Inside the *HelloWorld* project, expand the *Sources* folder, and double click on "main.cpp". You will see that a hello world program has already been written for you!

Replace that one with the following:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

To build your project, press *Ctrl-F9*, or go to *Build menu* > *Build*. If all goes well, you should see the following appear in the Build log window:

```
-------------- Build: Debug in HelloWorld (compiler: GNU GCC Compiler)---------------
mingw32-g++.exe -Wall -fexceptions -g -std=c++14  -c C:\CBProjects\HelloWorld\main.cpp -o obj\Debug\main.o
mingw32-g++.exe  -o bin\Debug\HelloWorld.exe obj\Debug\main.o
Output file is bin\Debug\HelloWorld.exe with size 1.51 MB
Process terminated with status 0 (0 minute(s), 0 second(s))
0 error(s), 0 warning(s) (0 minute(s), 0 second(s))
```

This means your compile was successful!

To run your compiled program, press *Ctrl-F10*, or go to *Build menu* > *Run*. You will see something similar to the following:

That is the result of your program!

> **For Linux users**
>
> Linux users may need to install the additional packages before Code::Blocks will compile. Please see the Code::Blocks installation instructions in lesson 0.6 – Installing an Integrated Development Environment (IDE) for more info.

Creating a project in VS Code

To start a new project, go to the *View* > *Explorer* menu (or press *Ctrl-Shift-E*). This will open the explorer pane. If you haven't previously opened a project, you should see an *Open Folder* button in the explorer pane – press it. If there is already an open project and you want to start a new one, choose *File* > *Open Folder* from the top nav.

Inside the dialog that opens, create a new folder named *HelloWorld* and then select this folder. This folder will be your project folder.

Next, we need to create the file that will contain our source code. Choose *File* > *New File* from the top nav, or click the *New File icon* to the right of *HELLOWORLD* in the explorer pane.

Name your file *main.cpp* and add the following contents to it:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

To compile *main.cpp* and run the program, make sure *main.cpp* is open in the main pane, and then either choose *Run* > *Run Without*

Debugging from the top nav, or click the *v* to the right of the play icon to the right of *main.cpp* tab and choose *Run C/C++ File*.

Next, choose the *g++ build and debug active file* option (macOS users should choose *clang++* instead of *g++*). Switch the tab from *DEBUG CONSOLE* to *TERMINAL* at the bottom of the window.

If the terminal contains the text "Hello, world!", then congratulations, you've just run your first C++ program!

If you're using g++ on the command line

In this case, you don't need to create a project. Simply paste the following into a text file named HelloWorld.cpp and save your file:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

From the command line, type:

`g++ -o HelloWorld HelloWorld.cpp`

This will compile and link HelloWorld.cpp. To run it, type:

`HelloWorld` (or possibly `./HelloWorld`), and you will see the output of your program.

If you're using other IDEs or a web-based compiler

You will have to figure out how to do the following on your own:

1. Create a console project (IDEs only)
2. Add a .cpp file to the project (IDEs only, if one isn't auto-created for you)
3. Paste the following code into the file:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

4. Compile the project
5. Run the project

If compiling fails

It's okay, take a deep breath. We can probably fix it. :)

First, look at the error message that the compiler gave you. Most often, it will contain a line number indicating which line was in error. Examine both that line and the lines around it, and make sure there are no typos or misspellings. Also make sure you're not including line numbers in your code (your editor should be providing those).

Second, look at the Q&A in lesson 0.8 – A few common C++ problems, as your issue may be covered there.

Third, read the comments below the lesson containing the example you're compiling – someone may have encountered the same issue and provided a solution.

Finally, if all of the above fail, try searching for your error message on your favorite search engine. It's likely someone else has encountered

this issue before and figured out how to fix it.

If your program runs but the console window flashes and closes immediately

When a console program is run, the console window will open and any output from the program will be written into the console window.

When the program has finished running, most modern IDEs will keep the console open (until you press a key) so you can inspect the results of the program before continuing. However, some older IDEs will automatically close the console window when the program finishes running. This is generally not what you want.

If your IDE closes the console window automatically, the following two steps can be used to ensure the console pauses at end of the program.

First, add or ensure the following lines are near the top of your program:

```
#include <iostream>
#include <limits>
```

Second, add the following code at the end of the main() function (just before the return statement):

```
std::cin.clear(); // reset any error flags
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // ignore any characters in the input b
until we find a newline
std::cin.get(); // get one more char from the user (waits for user to press enter)
```

This will cause your program to wait for the user to press enter before continuing (you may have to press enter twice), which will give you time to examine your program's output before your IDE closes the console window.

Other solutions, such as the commonly suggested `system("pause")` solution may only work on certain operating systems and should be avoided.

If the console window doesn't open at all and your program doesn't appear to be running, your

Configuring Your Compiler: Build Configurations

A **build configuration** (also called a **build target**) is a collection of project settings that determines how your IDE will build your project. The build configuration typically includes things like what the executable will be named, what directories the IDE will look in for other code and library files, whether to keep or strip out debugging information, how much to have the compiler optimize your program, etc… Generally, you will want to leave these settings at their default values unless you have a specific reason to change something.

When you create a new project in your IDE, most IDEs will set up two different build configurations for you: a release configuration, and a debug configuration.

The **debug configuration** is designed to help you debug your program, and is generally the one you will use when writing your programs. This configuration turns off all optimizations, and includes debugging information, which makes your programs larger and slower, but much easier to debug. The debug configuration is usually selected as the active configuration by default. We'll talk more about debugging techniques in a later lesson.

The **release configuration** is designed to be used when releasing your program to the public. This version is typically optimized for size and performance, and doesn't contain the extra debugging information. Because the release configuration includes all optimizations, this mode is also useful for testing the performance of your code (which we'll show you how to do later in the tutorial series).

When the *Hello World* program (from lesson 0.7 – Compiling your first program) was built using Visual Studio, the executable produced in the debug configuration was 65KB, whereas the executable built in the release version was 12KB. The difference is largely due to the extra debugging information kept in the debug build.

Although you can create your own custom build configurations, you'll rarely have a reason to unless you want to compare two builds made using different compiler settings.

> **Best practice**
>
> Use the *debug* build configuration when developing your programs. When you're ready to release your executable to others, or want to test performance, use the *release* build configuration.

Some IDEs (e.g. Visual Studio) also create separate build configurations for different platforms. For example, Visual Studio creates build configurations for both the x86 (32-bit) and the x64 (64-bit) platforms.

Switching between build configurations

For Visual Studio users

There are multiple ways to switch between *debug* and *release* in Visual Studio. The easiest way is to set your selection directly from the *Solution Configurations* dropdown in the *Standard Toolbar Options*:

Set it to *Debug* for now.

You can also access the configuration manager dialog by selecting *Build menu > Configuration Manager*, and change the *active solution configuration*.

To the right of the *Solutions Configurations* dropdown, Visual Studio also has a *Solutions Platform* dropdown that allows you to switch between x86 (32-bit) and x64 (64-bit) platforms.

For Code::Blocks users

In Code::Blocks, you should see an item called *Build Target* in the *Compiler toolbar*:

Set it to *Debug* for now.

For gcc and Clang users

Add `-ggdb` to the command line for debug builds and `-O2 -DNDEBUG` for release builds. Use the debug build options for now.

For GCC and Clang, the `-O#` option is used to control optimization settings. The most common options are as follows:

- `-O0` is the recommended optimization level for debug builds, as it disables optimization. This is the default setting.

- `-O2` is the recommended optimization level for release builds, as it applies optimizations that should be beneficial for all programs.
- `-O3` adds additional optimizations that may or may not perform better than `-O2` depending on the specific program. Once your program is written, you can try compiling your release build with `-O3` instead of `-O2` and measure to see which is faster.

See https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html for information on optimization options.

For VS Code users

When you first ran your program, a new file called *tasks.json* was created under the *.vscode* folder in the explorer pane. Open the *tasks.json* file, find *"args"*, and then locate the line *"${file}"* within that section.

Above the *"${file}"* line, add a new line containing the following command (one per line) when debugging: `"-ggdb",`

Above the *"${file}"* line, add new lines containing the following commands (one per line) for release builds: `"-O2",` `"-DNDEBUG",`

Modifying build configurations

In the next few lessons, we'll show you how to tweak some settings in your build configurations. Whenever changing a project setting, we recommend making the change in all build configurations.

This will help prevent making the change to the wrong build configuration, and ensure the change is still applied if you happen to switch build configurations later.

> **Tip**
>
> Whenever you update your project settings, make the change for all build configurations (unless it's not appropriate for some reason).

Configuring Your Compiler: Choosing a Language Standard

With many different versions of C++ available (C++98, C++03, C++11, C++14, C++17, C++20, C++23, etc…) how does your compiler know which one to use? Generally, a compiler will pick a standard to default to. Typically the default is *not* the most recent language standard – many default to C++14, which is missing many of the latest and greatest features.

If you wish to use a different language standard (and you probably will), you'll have to configure your IDE/compiler to do so.

The conventional names for language standards (e.g. C++20) are based on the year the language standard was published (or expected to be published). Because the year of publication is not actually known until it is close, language standards that are early in development sometimes use a development name instead. For example, C++20 is also known as C++2a.

Publication Year	Formal Name	Conventional name	Development name	Notes
2011	ISO/IEC 14882:2011	C++11	C++0x	
2014	ISO/IEC 14882:2014	C++14	C++1y	
2017	ISO/IEC 14882:2017	C++17	C++1z	
2020	ISO/IEC 14882:2020	C++20	C++2a	
2024	ISO/IEC 14882:2024	C++23	C++2b	Finalized (technically complete) in 2023
TBD	TBD	C++26	C++2c	

> **As an aside…**
>
> C++11 has development name `C++0x` because it was originally expected to be published before 2010.

Which language standard should you choose?

In professional environments, it's common to choose a language standard that is one or two versions back from the latest finalized standard (e.g. if C++20 were the latest finalized version, that means C++14 or C++17). This is typically done to ensure the compiler makers have had a chance to resolve defects, and so that best practices for new features are well understood. Where relevant, this also helps ensure better cross-platform compatibility, as compilers on some platforms may not provide full support for newer language standards immediately.

For personal projects and while learning, we recommend choosing the latest finalized standard, as there is little downside to doing so.

> **Tip**
>
> Many compilers offer "experimental" or "preview" support for features from the upcoming non-finalized language standards. Your IDE may offer this as an option in the settings where you configure your language standard, or you may be able to enable it manually using the upcoming language standard's conventional name or development name.
>
> Support for features from non-finalized language standards will be incomplete and possibly buggy.

> **Author's note**
>
> This website currently targets the C++17 standard, meaning our lessons and examples assume your compiler is C++17 capable. Some C++20 and C++23 content is available for those with compatible compilers.
>
> To take full advantage of all the lesson content, we recommend using the latest language standard your compiler supports.
>
> If your compiler doesn't support C++17, we recommend upgrading to one that does, or consider using an online compiler that supports C++17 or newer while learning.

> **A reminder**

> When changing your language standard (or any other project setting), make the change to all build configurations.

Setting a language standard in Visual Studio

As of the time of writing, Visual Studio 2022 defaults to C++14 capabilities, which does not allow for the use of newer features introduced in C++17 and C++20.

To use these newer features, you'll need to enable a newer language standard. Unfortunately, there is currently no way to do this globally – you must do so on a project-by-project basis.

> **Warning**
>
> With Visual Studio, you will need to reselect your language standard every time you create a new project.

To select a language standard, open your project, then go to *Project menu > (Your application's Name)* Properties, then open *Configuration Properties > C/C++ > Language*.

First, make sure the *Configuration* is set to "All Configurations".

From there, you can set the *C++ Language Standard* to the version of C++ you wish to use.

> **Tip**
>
> We recommend choosing the latest standard "ISO C++ Latest (/std:c++latest)", which will ensure you can use as many features as your compiler supports.

Make sure you're selecting the language standard from the dropdown menu (don't type it out).

> **Related content**
>
> For more information on Visual Studio language standard settings, Microsoft has a Visual Studio language standard reference document.

Setting a language standard in Code::Blocks

Code::Blocks may default to a pre-C++11 language standard. You'll definitely want to check and ensure a more modern language standard is enabled.

The good news is that Code::Blocks allows setting your language standard globally, so you can set it once (rather than per-project). To do so, go to *Settings menu > Compiler...*:

Then find the checkboxes labeled *Have g++ follow the C++XX ISO C++ language standard [-std=c++XX]*, where XX is some number (e.g. 20, 17, etc...) representing a language standard:

If C++23, C++20, or C++17 appears in this list, select the one that represents the latest ISO standard (e.g. select *Have g++ follow the C++20 ISO language standard*). If you see GNU standards in this list as well, ignore them.

If you do not see C++17 in this list, upgrade to the latest version of Code::Blocks.

> **Warning**
>
> As of the time of writing, the current version of Code::Blocks (20.03) is using an outdated compiler (GCC 8.1.0).
>
> We recommend updating your compiler by updating MinGW. The procedure to do so can be found under the Code::Blocks section in lesson 0.6 – Installing an Integrated Development Environment (IDE).

> **Tip**
>
> After updating to the latest version of MinGW, you can see if newer language standard checkboxes appear in the *Settings menu >*

> *Compiler...* menu. If not, you can manually select a C++ version as follows:
>
> - Find the "Other compiler options tab" of the "Global Compiler Settings" dialog.
> - Add one of the following compiler options: `-std=c++11`, `-std=c++14`, `-std=c++17`, `-std=c++20`, or `-std=c++23` (to enable C++11/14/17/20/23 support respectively). You can also try the latest code name (e.g. `-std=c++2c`) for experimental support for features from the upcoming language standard.

Setting a language standard in GCC/G++/Clang

For GCC/G++/Clang, you can use compiler options `-std=c++11`, `-std=c++14`, `-std=c++17`, `-std=c++20`, or `-std=c++23` (to enable C++11/14/17/20/23 support respectively). If you have GCC 8 or 9, you'll need to use `-std=c++2a` for C++20 support instead. You can also try the latest code name (e.g. `-std=c++2c`) for experimental support for features from the upcoming language standard.

Setting a language standard for VS Code

For VS Code, you can follow the rules above for setting a language standard in GCC/G++/Clang.

Place the appropriate language standard flag (including the double quotes and comma) in the `tasks.json` configuration file, in the `"args"` section, on its own line before `"${file}"`.

We also want to configure Intellisense to use the same language standard. For C++20, in `settings.json`, change or add the following setting on its own line: `"C_Cpp.default.cppStandard": "c++20"`.

What language standard is my compiler currently using?

This has been moved to the next lesson. See 0.13 – What language standard is my compiler using?.

Exporting your configuration

Having to reselect all of your settings options every time you create a new project is burdensome. Fortunately, most IDEs provide a way to export your settings. This is typically done by creating a new project template with the settings you want, and then selecting that project template when you create a new project.

For Visual Studio users

In Visual Studio, this option is available via Project -> Export Template. Select "Project template", add a name and optional description (e.g. C++20 console application), and then click "Finish".

Next time you create a new project, you'll see this template show up in your list of project templates.

Once you create a new project with this template, it may not open any files. You can open up your .cpp file in the Solution Explorer window by going to Solution -> <Project Name> -> Source Files -> <template name>.cpp.

Feb 2025: Reader Mingtao Yue kindly left a comment providing some workarounds to a couple of bugs in this feature.

For Code::Blocks users

In Code::Blocks, choose File -> Save project as template. Give your template a title, and save.

When you create a new project, you will find this template under the "User templates" option.

Where can I view the C++ standards document?

Each C++ language standard is described by a **standards document**, which is a formal technical document that is the authoritative source for the rules and requirements of a given language standard. The standards document is not designed for learning – rather, it's designed for compiler writers to be able to implement new language standards accurately. You will occasionally see people quoting the standards document when explaining how something works.

The approved C++ standards document for a given language standard is not available for free. There is a link to purchase the latest standard here.

When a new language standard is being developed, draft standards documents are published for review. These drafts *are* available online for free. The last draft standard before the approved standard is generally close enough to the official standard to use for most purposes. You can find the draft standards here.

Compilers often have incomplete support for new language features

Even after a language standard is finalized, compilers supporting that language standard often still have missing, partial, or buggy support for certain features.

If you attempt to compile a program that should compile but mysteriously won't, it's likely because of one of two reasons:

- Your compiler is configured to use an older language standard that doesn't support the feature you are using. Try selecting a newer language standard and compile your program again. You can use the program in lesson 0.13 – What language standard is my compiler using? to validate that you have configured your language standard selection correctly.
- The version of the compiler you are running may have missing, partial, or buggy support for a feature you are using.

The CPPReference website tracks compiler support for each feature per language standard. You can find those support tables linked from their home page, top right, under "Compiler Support" (by language standard). For example, you can see which C++23 features are supported here.

If a newer version of your compiler supports the problematic feature, you can upgrade your compiler to the latest version and try compiling your program again. If the latest version of your compiler still doesn't support the feature you are trying to use, you can either try a different compiler that does provide support, or find a solution using a different set of features.

Configuring your compiler: Compiler extensions

The C++ standard defines rules about how programs should behave in specific circumstances. And in most cases, compilers will follow these rules. However, many compilers implement their own changes to the language, often to enhance compatibility with other versions of the language (e.g. C99), or for historical reasons. These compiler-specific behaviors are called **compiler extensions**.

Writing a program that makes use of a compiler extension allows you to write programs that are incompatible with the C++ standard. Programs using non-standard extensions generally will not compile on other compilers (that don't support those same extensions), or if they do, they may not run correctly.

Frustratingly, compiler extensions are often enabled by default. This is particularly damaging for new learners, who may think some behavior that works is part of official C++ standard, when in fact their compiler is simply over-permissive.

Because compiler extensions are never necessary, and cause your programs to be non-compliant with C++ standards, we recommend turning compiler extensions off.

> **Best practice**
>
> Disable compiler extensions to ensure your programs (and coding practices) remain compliant with C++ standards and will work on any system.

Disabling compiler extensions

For Visual Studio users

To disable compiler extensions, right click on your project name in the *Solution Explorer* window, then choose *Properties*:

From the *Project* dialog, first make sure the *Configuration* field is set to *All Configurations*.

Then, click *C/C++ > Language tab*, and set *Conformance mode* to *Yes (/permissive-)* (if it is not already set to that by default).

For Code::Blocks users

Disable compiler extensions via *Settings menu > Compiler > Compiler flags tab*, then find and check the *-pedantic-errors* option.

For gcc and Clang users

You can disable compiler extensions by adding the *-pedantic-errors* flag to the compile command line.

For VS Code users

- Open the tasks.json file, find `"args"`, and then locate the line `"${file}"` within that section.
- Above the `"${file}"` line, add a new line containing the following commands:

```
"-pedantic-errors",
```

As of the time of writing, VS Code does not automatically add a newline to the end of code files that are missing it (something that is pedantically required by the C++ standard). Fortunately, we can tell VS Code to do so:

- Open VS Code and go to *File (Code if using a Mac) > Preferences > Settings*. This will open a settings dialog.
- Enter `insert final newline` into the search bar.
- In both the *Workspace Settings* and *User Settings* tabs, ensure the checkbox labeled *Files: Insert Final Newline* is checked.

> **Related content**
>
> Xcode users can refer to Rory's comment, who kindly provided instructions.

A reminder

These settings are applied on a per-project basis. You need to set them every time you create a new project, or create a template project with those settings once and use that to create new projects.

Configuring your compiler: Warning and error levels

When you write your programs, the compiler will check to ensure you've followed the rules of the C++ language (assuming you've turned off compiler extensions, as per lesson 0.10 – Configuring your compiler: Compiler extensions). If you have done something that definitively violates the rules of the language, then your program is **ill-formed**.

In most cases, when the compiler encounters some kind of issue, it will emit a **diagnostic message** (often called a **diagnostic** for short). The C++ standard does not define how diagnostic messages should be categorized, worded, or how those issues should affect the compilation of the program. However, modern compilers have conventionally adopted the following:

- A **diagnostic error** (**error** for short) means the compiler has decided to halt compilation, because it either cannot proceed or deems the error serious enough to stop. Diagnostic errors generated by the compiler are often called **compilation errors**, **compiler errors**, or **compile errors**.
- A **diagnostic warning** (**warning** for short) means the compiler has decided not to halt compilation. In such cases, the issue is simply ignored, and compilation proceeds.

> **Key insight**
>
> Compilers determine whether a non-blocking issue is a warning or an error. While they usually align in their categorization, in some cases, compilers may not agree – with one compiler emitting an error and another compiler emitting a warning for the same issue.

To help you identify where the issue is, diagnostic messages typically contain both the filename and line number where the compiler found the issue, and some text about what was expected vs what was found. The actual issue may be on that line, or on a preceding line. Once you've addressed the issue causing the diagnostic, you can try compiling again to see if the associated diagnostic message is no longer generated.

In some cases, the compiler may identify code that does not violate the rules of the language, but that it believes could be incorrect. In such cases, the compiler may decide to emit a warning as a notice to the programmer that something seems amiss. Such issues can be resolved either by fixing the issue the warning is pointing out, or by rewriting the offending lines of code in such a way that the warning is no longer generated.

> **For advanced readers**
>
> We show an example of a statement that is technically legal but modern compilers find suspicious in lesson 7.7 – External linkage and variable forward declarations.

In rare cases, it may be necessary to explicitly tell the compiler to not generate a particular warning for the line of code in question. C++ does not support an official way to do this, but many individual compilers (including Visual Studio and GCC) offer solutions (via non-portable `#pragma` directives) to temporarily disable warnings.

> **Best practice**
>
> Don't let warnings pile up. Resolve them as you encounter them (as if they were errors). Otherwise a warning about a serious issue may be lost amongst warnings about non-serious issues.

The linker may also generate diagnostic errors if there is an issue that occurs when linking that cannot be resolved.

Increasing your warning levels

By default, most compilers will only generate warnings about the most obvious issues. However, you can request your compiler be more assertive about providing warnings, and it is generally a good idea to do so.

> **Best practice**
>
> Turn your warning levels up, especially while you are learning. The additional diagnostic information may help in identifying programming mistakes that can cause your program to malfunction.

For Visual Studio users

To increase your warning levels, right click on your project name in the *Solution Explorer* window, then choose *Properties*:

From the *Project* dialog, first make sure the *Configuration* field is set to *All Configurations*.

Then select *C/C++ > General tab* and set *Warning level* to *Level4 (/W4)*:

Note: Do not choose *EnableAllWarnings (/Wall)* or you will be buried in warnings generated by the C++ standard library.

Visual Studio disables signed/unsigned conversion warnings by default, and those are useful, so if you are using Visual Studio 2019 or newer, let's enable those:

- From *C/C++ > Command Line tab*, under *Additional Options*, add `/w44365`. This tells the compiler to enable signed/unsigned conversion warnings at warning level 4 (which you enabled above).
- From *C/C++ > External Includes tab*, set *External Header Warning Level* to *Level3 (/external:W3)*. This tells the compiler to compile standard library headers at warning level 3 (instead of 4) so that compiling those headers doesn't trigger this warning.

The "External Includes" tab isn't shown in the graphic above, but appears in VS Community 2019 or newer between the "Browse Information" and "Advanced" tabs. See this link, which contains a recent photo of the dialog containing the "External Includes" tab.

If the above has been set correctly, compiling the following program should generate warning C4365:

```
void foo(int)
{
}

int main()
{
    unsigned int x { 5 };
    foo(x);

    return 0;
}
```

If you do not see the warning, check both the *Output* and *Error List* tabs (if they exist).

For Code::Blocks users

From *Settings menu > Compiler > Compiler settings tab*, find and check the options that correlate with *-Wall*, *-Weffc++*, and *-Wextra*:

Then go to the *Other compiler options tab*, and add *-Wconversion -Wsign-conversion* to the following text edit area:

Note: The *-Werror* parameter is explained below.

For gcc users

Add the following flags to your command line: *-Wall -Weffc++ -Wextra -Wconversion -Wsign-conversion*

For VS Code users

Open the tasks.json file, find "args", and then locate the line *"${file}"* within that section.

Above the *"${file}"* line, add new lines containing the following commands (one per line):

```
"-Wall",
"-Weffc++",
"-Wextra",
"-Wconversion",
"-Wsign-conversion",
```

Treat warnings as errors

It is also possible to tell your compiler to treat all warnings as if they were errors (in which case, the compiler will halt compilation if it finds any warnings). This is a good way to enforce the recommendation that you should fix all warnings (if you lack self-discipline, which most of us do).

> **Best practice**
>
> Enable “Treat warnings as errors”. This will force you to resolve all issues causing warnings.

For Visual Studio users

To treat warnings as errors, right click on your project name in the *Solution Explorer* window, then choose *Properties*:

From the *Project* dialog, first make sure the *Configuration* field is set to *All Configurations*.

Then select *C/C++ > General tab* and set *Treat Warnings As Errors* to *Yes (/WX)*.

For Code::Blocks users

From *Settings menu > Compiler > Other compiler options tab*, add *-Werror* to the text edit area:

For gcc users

Add the following flag to your command line: *-Werror*

For VS Code users

In the `tasks.json` file, add the following flags before “${file}”, one per line:

```
        "-Werror",
```

Installing an Integrated Development Environment (IDE)

An **Integrated Development Environment (IDE)** is a piece of software designed to make it easy to develop, build, and debug your programs.

A typical modern IDE will include:

- Some way to easily load and save your code files.
- A code editor that has programming-friendly features, such as line numbering, syntax highlighting, integrated help, name completion, and automatic source code formatting.
- A basic build system that will allow you to compile and link your program into an executable, and then run it.
- An integrated debugger to make it easier to find and fix software defects.
- Some way to install plugins so you can modify the IDE or add capabilities such as version control.

Some C++ IDEs will install and configure a C++ compiler and linker for you. Others will allow you to plug in a compiler and linker of your choice (installed separately).

And while you could do all of these things separately, it's much easier to just install an IDE and be able to do all of these things from a single interface.

So let's install one!

Choosing an IDE

The obvious next question is, "which one?". Many IDEs are free (in price), and you can install multiple IDEs if you wish to try more than one. We'll recommend a few of our favorites below.

If you have some other IDE in mind, that's fine too. The concepts we show you in these tutorials should generally work for any decent modern IDE. However, various IDEs use different names, layouts, key mappings, etc… so you may have to do a bit of searching in your IDE to find the equivalent functionality.

> **Tip**
>
> To get the most value out of this site, we recommend installing an IDE that comes with a compiler that supports at least C++17.
>
> If you're restricted to using a compiler that only supports C++14 (due to educational or business constraints), many of the lessons and examples will still work. However, if you encounter a lesson that uses capabilities from C++17 (or newer) and you're using an older language compiler, you'll have to skip it or translate it to your version, which may or may not be easy.
>
> You should not be using any compiler that does not support at least C++11 (which is typically considered the modern minimum spec for C++).
>
> We recommend installing the newest version of a compiler. If you can't use the newest version, these are the absolute minimum compiler versions with C++17 support:
>
> - GCC/G++ 7
> - Clang++ 8
> - Visual Studio 2017 15.7

Visual Studio (for Windows)

If you are developing on a Windows 10 or 11 machine, then we strongly recommend downloading Visual Studio 2022 Community.

Once you run the installer, you'll eventually come to a screen that asks you what workload you'd like to install. Choose *Desktop development with C++*. If you do not do this, then C++ capabilities will not be available.

The default options selected on the right side of the screen should be fine, but please ensure that the *Windows 11 SDK* (or *Windows 10 SDK* if that is the only one available) is selected. Windows 11 SDK apps can run on Windows 10.

Code::Blocks (for Linux or Windows)

If you are developing on Linux (or you are developing on Windows but want to write programs that you can easily port to Linux), we recommend Code::Blocks. Code::Blocks is a free, open source, cross-platform IDE that will run on both Linux and Windows.

For Windows users

Make sure to get the version of Code::Blocks that has MinGW bundled (it should be the one whose filename ends in *mingw-setup.exe*). This will install MinGW, which includes a Windows port of the GCC C++ compiler.

Code::Blocks 20.03 ships with an outdated version of MinGW that only supports C++17 (currently one version back from the latest version of C++). If you want to use the latest version of C++ (C++20), you will need to update MinGW. To do so, follow this procedure:

1. Install Code::Blocks as per the above.
2. Close Code::Blocks if it is open.
3. Open Windows File Explorer (Keyboard shortcut *Win-E*).
4. Navigate to your Code::Blocks install directory (probably C:Files (x86)).
5. Rename the "MinGW" directory to "MinGW.bak" (in case something goes wrong).
6. Open a browser and navigate to https://winlibs.com/.
7. Download an updated version of MinGW-w64. You probably want the one under *Release Versions -> UCRT Runtime -> LATEST -> Win64 -> without LLVM/Clang/LLD/LLDB -> Zip archive*.
8. Extract the "mingw64" folder to your Code::blocks install directory.
9. Rename "mingw64" to "MinGW".

Once you have confirmed the updated compiler works, you can delete the old folder ("MinGW.bak").

For Linux users

Some Linux installations may be missing dependencies needed to run or compile programs with Code::Blocks.

Debian-based Linux users (such as those on Mint or Ubuntu) may need to install the *build-essential* package. To do so from the terminal command line, type: `sudo apt-get install build-essential`.

Arch Linux users may need to install the *base-devel* package.

Users on other Linux variants will need to determine what their equivalent package manager and packages are.

When you launch Code::Blocks for the first time, you may get a *Compilers auto-detection* dialog. If you do, make sure *GNU GCC Compiler* is set as the default compiler and then select the *OK* button.

Q: What do I do if I get a "Can't find compiler executable in your configured search paths for GNU GCC Compiler" error?

Try the following:

1. If you're on Windows, make sure you've downloaded the version of Code::Blocks WITH MinGW. It's the one with "mingw" in the name.
2. Try going to settings, compiler, and choose "reset to defaults".
3. Try going to settings, compiler, toolchain executables tab, and make sure "Compiler's installation directory" is set to the MinGW directory (e.g. C:Files (x86)).
4. Try doing a full uninstall, then reinstall.
5. Try a different compiler.

Visual Studio Code (for experienced Linux, macOS, or Windows users)

Visual Studio Code (also called "VS Code", not to be confused with the similarly named "Visual Studio Community") is a code editor that is a popular choice with experienced developers because it is fast, flexible, open source, works for multiple programming languages, and is available for many different platforms.

The downside is that VS Code is much harder to configure correctly than other choices on this list (and on Windows, harder to install as well). Before proceeding, we recommend reading through the installation and configuration documents linked below to ensure you understand and are comfortable with the steps involved.

> **Warning**
>
> This tutorial series does not have complete instructions for VS Code.
>
> Visual Studio Code is *not* a good option for C++ beginners, and readers have reported many different challenges getting Visual Studio Code installed and configured correctly for C++.
>
> Please do not choose this option unless you are already familiar with Visual Studio Code from prior use, or have experience debugging configuration issues.
>
> We cannot provide installation or configuration support for Visual Studio Code on this site.

A tip o' the hat to user glibg10b for providing an initial draft of VS Code instructions across multiple articles.

For Linux users

VS Code should be downloaded using your distribution's package manager. The VS Code instructions for linux cover how to do this for various Linux distributions.

Once VS Code is installed, follow the instructions on how to configure C++ for linux.

For Mac users

The VS Code instructions for Mac detail how to install and setup VS Code for macOS.

Once VS Code is installed, follow the instructions on how to configure C++ for Mac.

For Windows users

The VS Code instructions for Windows detail how to install and setup VS Code for Windows.

Once VS Code is installed, follow the instructions on how to configure C++ for Windows.

Other macOS IDEs

Other popular Mac choices include Xcode (if it is available to you) and the Eclipse code editor. Eclipse is not set up to use C++ by default, and you will need to install the optional C++ components.

Other compilers or platforms

Q: Can I use a web-based compiler?

Yes, for some things. While your IDE is downloading (or if you're not sure you want to commit to installing one yet), you can continue this tutorial using a web-based compiler. We recommend one of the following:

- TutorialsPoint
- Wandbox (can choose different versions of GCC or Clang)
- Godbolt (can see assembly)

Web-based compilers are fine for dabbling and simple exercises. However, they are generally quite limited in functionality – many won't allow you to create multiple files or effectively debug your programs, and most don't support interactive input. You'll want to migrate to a full IDE when you can.

Q: Can I use a command-line compiler (e.g. g++ on Linux)?

Yes, but we don't recommend it for beginners. You'll need to find your own editor and look up how to use it elsewhere. Learning to use a command line debugger is not as easy as an integrated debugger, and will make debugging your programs more difficult.

Q: Can I use other code editors or IDEs, such as Eclipse, Sublime, or Notepad++?

Yes, but we don't recommend it for beginners. There are many great code editors and IDEs that can be configured to support a wide variety of languages, and allow you to mix and match plugins to customize your experience however you like. However, many of these editors and IDEs require additional configuration to compile C++ programs, and there's a lot that can go wrong during that process. For beginners, we recommend something that works out of the box, so you can spend more time learning to code and less time trying to figure out why your code editor isn't working properly with your compiler or debugger.

IDEs to avoid

You should avoid the following IDEs altogether because they do not support at least C++11, do not support C++ at all, or are no longer actively supported or maintained:

- Borland Turbo C++ – does not support C++11
- Visual Studio for Mac – does not support C++. (Note: this is a different product than VS Code).
- Dev C++ – not actively supported

There is no good reason to use an outdated or unsupported compiler when lightweight, free alternatives that support modern C++ exist.

When things go wrong (a.k.a. when IDE stands for "I don't even...")

IDE installations seem to cause their fair share of problems. Installation might fail outright (or installation might work but the IDE will have problems when you try to use it due to a configuration issue). If you encounter such issues, try uninstalling the IDE (assuming it installed in the first place), reboot your machine, disable your antivirus or anti-malware temporarily, and try the installation again.

If you're still encountering issues at this point, you have two options. The easier option is to try a different IDE. The other option is to fix the problem. Unfortunately, the causes of installation and configuration errors are varied and specific to the IDE software itself, and we're unable to effectively advise on how to resolve such issues. In this case, we recommend copying the error message or problem you are having into your favorite search engine (such as Google or Duck Duck Go) and trying to find a forum post elsewhere from some poor soul who has inevitably encountered the same issue. Often there will be suggestions on things you can try to remedy the issue.

Moving on

Once your IDE is installed (which can be one of the hardest steps if things don't go as expected), or if you're temporarily proceeding with a web-based compiler, you are ready to write your first program!

Before C++, there was C

The C language was developed in 1972 by Dennis Ritchie at Bell Telephone laboratories, primarily as a systems programming language (a language to write operating systems with). Ritchie's primary goals were to produce a minimalistic language that was easy to compile, allowed efficient access to memory, produced efficient code, and was self-contained (not reliant on other programs). For a high-level language, C was designed to give the programmer a lot of control, while allowing developers to write a program that could be run on different platforms.

C ended up being so efficient and flexible that in 1973, Ritchie and Ken Thompson rewrote most of the Unix operating system using C. Many previous operating systems had been written in assembly. Unlike assembly, which produces programs that can only run on specific CPUs, C has excellent portability, allowing Unix to be easily recompiled on many different types of computers and speeding its adoption. C and Unix had their fortunes tied together, and C's popularity was in part tied to the success of Unix as an operating system.

In 1978, Brian Kernighan and Dennis Ritchie published a book called "The C Programming Language". This book, which was commonly known as K&R (after the authors' last names), provided an informal specification for the language and became a de facto standard. When maximum portability was needed, programmers would stick to the recommendations in K&R, because most compilers at the time were implemented to K&R standards.

In 1983, the American National Standards Institute (ANSI) formed a committee to establish a formal standard for C. In 1989 (committees take forever to do anything), they finished, and released the C89 standard, more commonly known as ANSI C. In 1990 the International Organization for Standardization (ISO) adopted ANSI C (with a few minor modifications). This version of C became known as C90. Compilers eventually became ANSI C/C90 compliant, and programs desiring maximum portability were coded to this standard.

In 1999, the ISO committee released a new version of C informally named C99. C99 adopted many features which had already made their way into compilers as extensions, or had been implemented in C++.

C++

C++ (pronounced "see plus plus") was developed by Bjarne Stroustrup at Bell Labs as an extension to C, starting in 1979. C++ adds many

new features to the C language, and is perhaps best thought of as a superset of C, though this is not strictly true (as C99 introduced a few features that do not exist in C++). C++'s most notable innovation over C is that it supports object-oriented programming. As for what an "object" is and how it differs from traditional programming methods, well, we'll cover that in later chapters.

C++ was standardized in 1998 by the ISO committee. This means the ISO standards committee approved a document (called a **standards document**) that provides a formal description of the C++ language. The goal of standardization is to help ensure that C++ code behaves consistently across different compilers and platforms.

A minor update to the language was published in 2003 (informally named C++03).

Five major updates to the C++ language (informally named C++11, C++14, C++17, C++20, and C++23) have been made since then, each adding additional functionality. C++11 in particular added a huge number of new capabilities, and is widely considered to be the new baseline version of the language. Future upgrades to the language are expected every three or so years.

Because the official name of the approved standards is complex (C++20's formal name is ISO/IEC 14882:2020), standards are conventionally referred to by informal names, which include the last two digits of the year of publication (or expected publication). For example, C++20 refers to the version of the language published in 2020.

C and C++'s philosophy

The underlying design philosophy of C and C++ can be summed up as "trust the programmer" – which is both wonderful and dangerous. C++ is designed to allow the programmer a high degree of freedom to do what they want. However, this also means the language often won't stop you from doing things that don't make sense, because it will assume you're doing so for some reason it doesn't understand. There are quite a few pitfalls that new programmers are likely to fall into if caught unaware. This is one of the primary reasons why knowing what you shouldn't do in C/C++ is almost as important as knowing what you should do.

Q: What is C++ good at?

C++ excels in situations where high performance and precise control over memory and other resources is needed. Here are a few types of applications that C++ would excel in:

- Video games
- Real-time systems (e.g. for transportation, manufacturing, etc…)
- High-performance financial applications (e.g. high frequency trading)
- Graphical applications and simulations
- Productivity / office applications
- Embedded software
- Audio and video processing
- Artificial intelligence and neural networks

C++ also has a large number of high-quality 3rd party libraries available, which can shorten development times significantly.

Q: Isn't C++ dying?

Nope. Surveys consistently indicate that C++ is the 2nd or 3rd most popular compiled language (behind Java and sometimes C#, and just ahead of C), and the 5th or 6th most popular language overall (excluding HTML, SQL, and shell scripting languages).

C++ is one of the most popular languages for learning to code, owing to the abundance of teaching resources, the large community, and the number of college courses that teach it.

With language updates every three years, a huge number of useful third party libraries, and dominance in the ever-popular video game industry, C++ continues to thrive.

Q: Do I need to know C before I do these tutorials?

Nope! It's perfectly fine to start with C++, and we'll teach you everything you need to know (including pitfalls to avoid) along the way.

Once you know C++, it should be pretty easy to learn standard C if you ever have the need. These days, C is mostly used for niche use

cases: code that runs on embedded devices, when you need to interact with other languages that can only interface with C, etc... For most other cases, C++ is recommended.

Introduction to C++ Development

Before we can write and execute our first C++ program, we need to understand in more detail how C++ programs get developed. Here is a graphic outlining a simplistic approach:

Step 1: Define the problem that you would like to solve

This is the "what" step, where you figure out what problem you are intending to solve. Coming up with the initial idea for what you would like to program can be the easiest step, or the hardest. But conceptually, it is the simplest. All you need is an idea that can be well defined, and you're ready for the next step.

Here are a few examples:

- "I want to write a program that will allow me to enter many numbers, then calculates the average."
- "I want to write a program that generates a 2d maze and lets the user navigate through it. The user wins if they reach the end."
- "I want to write a program that reads in a file of stock prices and predicts whether the stock will go up or down."

Step 2: Determine how you are going to solve the problem

This is the "how" step, where you determine how you are going to solve the problem you came up with in step 1. It is also the step that is most neglected in software development. The crux of the issue is that there are many ways to solve a problem – however, some of these solutions are good and some of them are bad. Too often, a programmer will get an idea, sit down, and immediately start coding a solution. This often generates a solution that falls into the bad category.

Typically, good solutions have the following characteristics:

- They are straightforward (not overly complicated or confusing).
- They are well documented (especially around any assumptions being made or limitations).
- They are built modularly, so parts can be reused or changed later without impacting other parts of the program.
- They can recover gracefully or give useful error messages when something unexpected happens.

When you sit down and start coding right away, you're typically thinking "I want to do <something>", so you implement the solution that gets you there the fastest. This can lead to programs that are fragile, hard to change or extend later, or have lots of bugs. A **bug** is any kind of programming error that prevents the program from operating correctly.

> **As an aside…**
>
> The term *bug* was first used by Thomas Edison back in the 1870s! However, the term was popularized in the 1940s when engineers found an actual moth stuck in the hardware of an early computer, causing a short circuit. Both the log book in which the error was reported and the moth are now part of the Smithsonian Museum of American History. It can be viewed here.

Various studies have shown that on complex software systems, only 10-40% of a programmer's time is actually spent writing the initial program. The other 60-90% is spent on maintenance, which can consist of **debugging** (removing bugs), updates to cope with changes in the environment (e.g. to run on a new OS version), enhancements (minor changes to improve usability or capability), or internal improvements (to increase reliability or maintainability)[1].

Consequently, it's worth your time to spend a little extra time up front (before you start coding) thinking about the best way to tackle a problem, what assumptions you are making, and how you might plan for the future, in order to save yourself a lot of time and trouble down the road.

We'll talk more about how to effectively design solutions to problems in a future lesson.

Step 3: Write the program

In order to write the program, we need two things: First, we need knowledge of a programming language – that's what these tutorials are for! Second, we need a text editor to write and save our C++ programs. The set of C++ instructions that we input into the text editor is called the program's **source code** (often shortened to just **code**). It's possible to write a program using any text editor you want, even

something as simple as Windows' notepad or Unix's vi or pico.

A program typed into a basic text editor would look something like this:

```
#include <iostream>

int main()
{
    std::cout << "Here is some text.";
    return 0;
}
```

However, we strongly urge you to use an editor that is designed for programming (called a **code editor**). Don't worry if you don't have one yet. We'll cover how to install a code editor shortly.

A typical editor designed for coding has a few features that make programming much easier, including:

1. Line numbering. Line numbering is useful when the compiler gives us an error, as a typical compiler error will state: *some error code/message, line 64*. Without an editor that shows line numbers, finding line 64 can be a real hassle.
2. Syntax highlighting and coloring. Syntax highlighting and coloring changes the color of various parts of your program to make it easier to identify the different components of your program.
3. An unambiguous, fixed-width font (often called a "monospace font"). Non-programming fonts often make it hard to distinguish between the number 0 and the letter O, or between the number 1, the letter l (lower case L), and the letter I (upper case i). A good programming font will ensure these symbols are visually differentiated in order to ensure one isn't accidentally used in place of the other. All code editors should have this enabled by default, but a standard text editor might not. Using a fixed-width font (where all symbols have the same width) makes it easier to properly format and align your code.

Here's an example of a C++ program with line numbering, syntax highlighting, and a fixed-width font:

```
#include <iostream>

int main()
{
    std::cout << "Here is some text.";
    return 0;
}
```

Note how much easier this is to understand than the non-highlighted version. The source code we show in this tutorial will have both line numbering and syntax highlighting to make that code easier to follow.

> **Tip**
>
> Coding Font and Programming Fonts both have neat tools that allow you to compare different coding fonts to see which ones you like best.

> **For advanced readers**
>
> Because source code is written using ASCII characters, programming languages use a certain amount of ASCII art to represent mathematical concepts. For example, `≠` is not part of the ASCII character set, so programming languages typically use `!=` to represent mathematical inequality instead.
>
> Some programming fonts, such as Fira Code, use ligatures to combine such "art" back into a single character. For example, instead of displaying `!=`, Fira Code will display `≠` (using the same width as the two-character version). Some people find this easier to read, others prefer sticking with a more literal interpretation of the underlying characters.

Many simple C++ programs only have one source code file, but complex C++ programs can have hundreds or even thousands of source

code files.

Each source code file in your program will need to be saved to disk, which means each source code file needs a filename. C++ does not have any requirements for naming files. However, the de-facto standard is to name the first/primary source file created for a program `main.cpp`. The filename (`main`) makes it easy to determine which is the primary source code file, and the `.cpp` extension indicates that the file is a C++ source code file.

You may occasionally see the first/primary source code file named after the name of the program instead (e.g. `calculator.cpp`, `poker.cpp`). You may also occasionally see other extensions used (e.g. `.cc` or `.cxx`).

> **Best practice**
>
> Name the first/primary source code file in each program `main.cpp`. This makes it easy to determine which source code file is the primary one.

Once you've written your program, the next steps are to convert the source code into something that can be run, and then see whether it works! We'll discuss those steps (4-7) in the next lesson.

Introduction to Programming Languages

Modern computers are incredibly fast, and getting faster all the time. However, computers also have some significant constraints: they only natively understand a limited set of instructions, and must be told exactly what to do.

A **computer program** is a sequence of instructions that directs a computer to perform certain actions in a specified order. Computer programs are typically written in a **programming language**, which is a language designed to facilitate the writing of instructions for computers. There are many different programming languages available, each of which caters to a different set of needs. The act (and art) of writing a program is called **programming**. We'll talk more specifically about how to create programs in C++ in upcoming lessons in this chapter.

When a computer is performing the actions described by the instructions in a computer program, we say it is **running** or **executing** the program. A computer will not begin execution of a program until told to do so. That typically requires the user to **launch** (or **run** or **execute**) the program, although programs may also be launched by other programs.

Programs are executed on the computer's **hardware**, which consists of the physical components that make up a computer. Notable hardware found on a typical computing device includes:

- A CPU (central processing unit, often called the "brain" of the computer), which actually executes the instructions.
- Memory, where computer programs are loaded prior to execution.
- Interactive devices (e.g. a monitor, touch screen, keyboard, or mouse), which allow a person to interact with a computer.
- Storage devices (e.g. a hard drive, SSD, or flash memory), which retain information (including installed programs) even when the computer is turned off.

In contrast, the term **software** broadly refers to the programs on a system that are designed to be executed on hardware.

In modern computing, programs often interact with more than just hardware – they also interact with other software on the system (particularly the operating system). The term **platform** refers to a compatible set of hardware and software (OS, browser, etc…) that provides an environment for software to run. For example, the term "PC" is used colloquially to mean the platform consisting of a Windows OS running on an x86-family CPU.

Platforms often provide useful services for the programs running on them. For example, a desktop application might request the operating system give them a chunk of free memory, create a file over there, or play a sound. The running program doesn't have to know how this is actually facilitated. If a program uses capabilities or services provided by the platform, it becomes dependent on that platform, and cannot be run on other platforms without modification. A program that can be easily transferred from one platform to another is said to be **portable**. The act of modifying a program so that it runs on a different platform is called **porting**.

Now that we've talked about programs, let's discuss programming languages. This isn't just a history lesson, we'll also be introducing terminology that will come up in future lessons.

Machine Language

A computer's CPU is incapable of understanding C++. Instead, CPUs are only capable of processing instructions written in **machine language** (or **machine code**). The set of all possible machine language instructions that a given CPU can understand is called an **instruction set**.

Here is a sample machine language instruction: `10110000 01100001`.

Each instruction is understood by the CPU as a command to do a very specific job, such as "compare these two numbers", or "copy this number into that memory location". Back when computers were first invented, programmers had to write programs directly in machine language, which was a very difficult and time-consuming thing to do.

How these instructions are organized and interpreted is beyond the scope of this tutorial series, but it is worth noting a few things.

First, each instruction is composed of a sequence of 1s and 0s. Each individual 0 or 1 is called a **binary digit**, or **bit** for short. The number of bits in a machine language instruction varies – for example, some CPUs process instructions that are always 32 bits long, whereas some other CPUs (such as those from the x86 family, which you may be using) have instructions that can be a variable length.

Second, each family of compatible CPUs (e.g. x86, Arm64) has its own machine language, and this machine language is not compatible with the machine language of other CPU families. This means machine language programs written for one CPU family cannot be run on CPUs from a different family!

> **Related content**
>
> A “CPU family” is formally called an “instruction set architecture” (“ISA” for short). Wikipedia has a list of different CPU families here.

Assembly Languages

Machine language instructions (like `10110000 01100001`) are ideal for a CPU, but are difficult for humans to understand. Since programs (at least historically) have been written and maintained by humans, it makes sense that programming languages should be designed with human needs in mind.

An **assembly language** (often called **assembly** for short) is a programming language that essentially functions as a more human-readable machine language. Here is the same instruction as above in x86 assembly language: `mov al, 0x61`.

> **Optional reading**
>
> This instruction illustrates many of the capabilities that make assembly more readable than machine language.
>
> - The operation (what the instruction does) is identified by a short mnemonic (typically a 3-5 letter name). `mov` is easily understood to be a mnemonic for “move”, which is an operation that copies bits from one location to another.
> - Registers (fast memory locations that are part of the CPU itself) are accessed by a name. `al` is the name of a specific register on an x86 CPU.
> - Numbers can be specified in a more convenient format. Assembly languages typically support both decimal numbers (e.g. `97`) and hexadecimal numbers (e.g. `0x61`).
>
> It is fairly easy to understand that the assembly instruction `mov al, 0x61` copies hexadecimal number `0x61` into the `al` CPU register.

Since CPUs do not understand assembly language, assembly programs must be translated into machine language before they can be executed. This translation is done by a program called an **assembler**. Because each assembly language instruction is typically designed to mirror an equivalent machine language instruction, the translation process is typically straightforward.

Just like each CPU family has its own machine language, each CPU family also has its own assembly language (which is designed to be assembled into machine language for that same CPU family). This means there are many different assembly languages. Although conceptually similar, different assembly languages support different instructions, use different naming conventions, etc…

Introduction to low-level languages

Machine languages and assembly languages are considered low-level languages, as these languages provide minimal abstraction from the architecture of the machine. In other words, the programming language itself is tailored to the specific instruction set architecture it will be run on.

Low-level languages have a number of notable downsides:

- Programs written in a low-level language are not portable. Since a low-level language is tailored to a specific instruction set architecture, the programs written in the language are too. Porting such programs to other architectures is typically non-trivial.
- Writing a program in a low-level language requires detailed knowledge of the architecture itself. For instance, the instruction `mov al, 061h` requires knowing that `al` refers to a CPU register available on this specific platform, and understanding how to work with that register. On a different architecture, this register might be named something different, have different limitations, or not exist at all.
- Low-level programs are hard to understand. While individual assembly instructions can be quite understandable, it can still be hard to deduce what a section of assembly code is actually doing. And since assembly programs require many instructions to do even simple tasks, they tend to be quite long.
- It is hard to write assembly programs of significant complexity because the language only provides primitive capabilities. The programmer is left to implement everything they need themselves.

The primary benefit of low-level languages is that they are fast. Assembly is still used today when there are sections of code that are performance critical. And it’s also used in a few other cases, one of which we’ll discuss in a moment.

Introduction to high-level Languages

To address many of the above downsides, new “high-level” programming languages such as C, C++, Pascal (and later, languages such as Java, Javascript, and Perl) were developed.

Here is the same instruction as above in C/C++: `a = 97;` .

Much like assembly programs (which must be assembled to machine language), programs written in a high-level language must be translated into machine language before they can be run. There are two primary ways this is done: compiling and interpreting.

C++ programs are usually compiled. A **compiler** is a program (or collection of programs) that reads the source code of one language (usually a high-level language) and translates it into another language (usually a low-level language). For example, a C++ compiler translates C++ source code into machine code.

> **Optional reading**
>
> Most C++ compilers can also be configured to generate assembly code. This is useful when a programmer wants to see what specific instructions the compiler is generating for a section of the program.

The machine code output by the compiler can then be packaged into an executable file (containing machine language instructions) that can distributed to others and launched by the operating system. Notably, running the executable file does not require the compiler to be installed.

In the beginning, compilers were primitive and produced slow, unoptimized assembly or machine code. However, over the years, compilers have become very good at producing fast, optimized code, and in some cases can do a better job than humans can!

Here is a simplified representation of the compiling process:

Alternatively, an **interpreter** is a program that directly executes the instructions in the source code without requiring them to be compiled first. Interpreters tend to be more flexible than compilers, but are less efficient when running programs because the interpreting process needs to be done every time the program is run. This also means the interpreter must be installed on every machine where an interpreted program will be run.

Here is a simplified representation of the interpretation process:

> **Optional reading**
>
> A good comparison of the advantages of compilers vs interpreters can be found here.
>
> Another advantage of compiled programs is that distributing a compiled program does not require distributing the source code. In a non-open-source environment, this is important for intellectual property (IP) protection purposes.

Most high-level languages can be either compiled or interpreted. Traditionally, high-level languages like C, C++, and Pascal are compiled, whereas “scripting” languages like Perl and Javascript tend to be interpreted. Some languages, like Java, use a mix of the two. We’ll explore C++ compilers in more detail shortly.

The benefits of high-level languages

High-level languages are named as such because they provide a high level of abstraction from the underlying architecture.

Consider the instruction `a = 97;` . This instruction lets us store the value `97` somewhere in memory, without needing to know exactly where that value will be placed, or what specific machine code instruction is needed by the CPU to store that value. In fact, there is nothing platform-specific about this instruction at all. The compiler does all the work to figure out how this C++ instruction translates into platform-specific machine code.

High-level languages allow programmers to write programs without knowing much about the platform it will be run on. This not only makes programs easier to write, it also makes them significantly more portable. If we’re careful, we can write a single C++ that will compile on every platform that has a C++ compiler! A program that is designed to run on multiple platforms is said to be **cross-platform**.

> **For advanced readers**

> The following is a partial list of things that can inhibit the portability of your C++ code:
>
> - Many operating systems, such as Microsoft Windows, offer platform-specific capabilities that you can use in your code. These can make it much easier to write a program for a specific operating system, or provide deeper integration with that operating system than would otherwise be possible.
> - Many third-party libraries are only available on certain platforms. If you use one of these, you will be limited to the platforms for which that library is supported.
> - Some compilers support compiler-specific extensions, which are capabilities that are only available in that compiler. If you use these, your programs won't be able to be compiled by other compilers that don't support the same extensions without modification. We'll talk more about these later, once you've installed a compiler.
> - In certain cases, the C++ language allows the compiler to determine how something should behave. We discuss this further in lesson 1.6 – Uninitialized variables and undefined behavior under "implementation-defined behavior".
>
> If you're only targeting a single platform, then portability may not matter that much. But many applications these days target multiple platforms in order to widen their reach. For example, a mobile app will probably want to target both iOS and Android.
>
> Even if portability doesn't seem useful initially, many applications that initially targeted a single platform (e.g. PC) decided to port to other platforms (e.g. Mac and various consoles) after seeing some level of success and interest. If you don't start with portability in mind, it will be more work to port your application later.
>
> In these tutorials, we will avoid platform-specific code as much as possible, so that our programs will run on any platform that has a modern C++ compiler.

High-level languages have other benefits as well:

- Programs written in a high-level language are easier to read, write, and learn because their instructions more closely resemble the natural language and mathematics that we use every day. In many cases, high-level languages require fewer instructions to perform the same tasks as low-level languages. For example, in C++ you can write `a = b * 2 + 5;` in one line. In assembly language, this would take 4 to 6 different instructions. This makes programs written using high-level languages more concise, which makes them easier to understand.
- High-level languages typically include additional capabilities that make it easier to perform common programming tasks, such as requesting a block of memory or manipulating text. For example, it only takes a single instruction to determine whether the characters "abc" exist within a large block of text (and if so, how many characters has to be examined until "abc" was found). This can dramatically reduce complexity and development times.

> **Nomenclature**
>
> Although C++ is technically considered a high-level language, newer programming languages (e.g. scripting languages) provide an even higher level of abstraction. As such, C++ is sometimes inaccurately called a "low-level language" in comparison.

> **Author's note**
>
> Today, C++ would probably be more accurately described as a mid-level language. However, this also highlights one of C++'s key strengths: it often provides the ability to work at different levels of abstraction. You can choose to operate at a lower level for better performance and precision, or at a higher level for greater convenience and simplicity.

Rules, Best practices, and warnings

As we proceed through these tutorials, we'll highlight many important points under the following three categories:

> **Rule**
>
> Rules are instructions that you *must* do, as required by the language. Failure to abide by a rule will generally result in your program not working.

> **Best practice**
>
> Best practices are things that you *should* do, because that way of doing things is either conventional (idiomatic) or recommended. That is, either everybody does it that way (and if you do otherwise, you'll be doing something people don't expect), or it is generally superior to the alternatives.

> **Warning**
>
> Warnings are things that you *should not* do, because they will generally lead to unexpected results.

Introduction to the Compiler, Linker, and Libraries

Continuing our discussion of this diagram from the previous lesson (0.4 – Introduction to C++ development):

Let's discuss steps 4-7.

Step 4: Compiling your source code

In order to compile C++ source code files, we use a C++ compiler. The C++ compiler sequentially goes through each source code (.cpp) file in your program and does two important tasks:

First, the compiler checks your C++ code to make sure it follows the rules of the C++ language. If it does not, the compiler will give you an error (and the corresponding line number) to help pinpoint what needs fixing. The compilation process will also be aborted until the error is fixed.

Second, the compiler translates your C++ code into machine language instructions. These instructions are stored in an intermediate file called an **object file**. The object file also contains other data that is required or useful in subsequent steps (including data needed by the linker in step 5, and for debugging in step 7).

Object files are typically named *name.o* or *name.obj*, where *name* is the same name as the .cpp file it was produced from.

For example, if your program had 3 .cpp files, the compiler would generate 3 object files:

C++ compilers are available for many different operating systems. We will discuss installing a compiler shortly, so there is no need to do so now.

Step 5: Linking object files and libraries and creating the desired output file

After the compiler has successfully finished, another program called the **linker** kicks in. The linker's job is to combine all of the object files and produce the desired output file (such as an executable file that you can run). This process is called **linking**. If any step in the linking process fails, the linker will generate an error message describing the issue and then abort.

First, the linker reads in each of the object files generated by the compiler and makes sure they are valid.

Second, the linker ensures all cross-file dependencies are resolved properly. For example, if you define something in one .cpp file, and then use it in a different .cpp file, the linker connects the two together. If the linker is unable to connect a reference to something with its definition, you'll get a linker error, and the linking process will abort.

Third, the linker typically links in one or more **library files**, which are collections of precompiled code that have been "packaged up" for reuse in other programs.

Finally, the linker outputs the desired output file. Typically this will be an executable file that can be launched (but it could be a library file if that's how you've set up your project).

The standard library

C++ comes with an extensive library called the **C++ Standard Library** (usually called "the standard library") that provides a set of useful capabilities for use in your programs. One of the most commonly used parts of the C++ standard library is the Input/Output library (often called "iostream"), which contains functionality for printing text on a monitor and getting keyboard input from a user.

Almost every C++ program written utilizes the standard library in some way, so it's extremely common to have the C++ standard library linked into your programs. Most C++ linkers are configured to link in the standard library by default, so this generally isn't something you need to worry about.

3rd party libraries

You can optionally link **third party libraries**, which are libraries that are created and distributed by independent entities (rather than as part of the C++ standard). For example, let's say you wanted to write a program that played sounds. The C++ standard library contains no such functionality. While you could write your own code to read in the sound files from disk, check to ensure they were valid, or figure out how to route the sound data to the operating system or hardware to play through the speaker – that would be a lot of work! Instead, you'd be more likely to find some existing software project that has a library that already implements all of these things for you.

We'll talk about how to link in libraries (and create your own!) in the appendix.

Building

Because there are multiple steps involved, the term **building** is often used to refer to the full process of converting source code files into an executable that can be run. A specific executable produced as the result of building is sometimes called a **build**.

> **For advanced readers**
>
> For complex projects, build automation tools (such as **make** or **build2**) are often used to help automate the process of building programs and running automated tests. While such tools are powerful and flexible, because they are not part of the C++ core language, nor do you need to use them to proceed, we'll not discuss them as part of this tutorial series.

Steps 6 & 7: Testing and Debugging

This is the fun part! You are now able to run your executable and see what it does!

Once you can run your program, then you can test it. **Testing** is the process of assessing whether your software is working as expected. Basic testing typically involves trying different input combinations to ensure the software behaves correctly in different cases.

If the program does not behave as expected, then you will have to do some **debugging**, which is the process of finding and fixing programming errors.

We will discuss how to test and debug your programs in more detail in future chapters.

Integrated development environments (IDEs)

Note that steps 3, 4, 5, and 7 all involve software programs that must be installed (editor, compiler, linker, debugger). While you can use separate programs for each of these activities, a software package known as an integrated development environment (IDE) bundles and integrates all of these features together. We'll discuss IDEs, and install one, in the next section.

Welcome!

Welcome to the Learn C++ tutorials! Above all else, these tutorials aim to make learning C++ easy.

Unlike many other sites and books, these tutorials don't assume you have any prior programming experience. We'll teach you everything you need to know as you progress, with *lots* of examples along the way.

Whether you're interested in learning C++ as a hobby or for professional development, you're in the right place!

A testimonial from reader Syam from Trinidad:

> It's crazy how perfect this website explains C++ concepts and teaches programming. I read the book "Programming: Principles and Practice Using C++" [by Bjarne Stroustrup, the creator of C++] and it was rather confusing.... This website makes it clear the first time, and they explain everything you need to know. Somehow they know what I don't know, and what I need to know. It's almost like my future self went back in the past to teach me what mistakes I will make.

Lesson structure

The lessons in this introductory chapter are aimed at giving you some context around what C++ is, how it came about, how programs work, and what software you need to install to create your own programs. You'll even write your own first program.

Further chapters will explore different parts of the C++ language. In the first chapter (chapter 1), you'll get a broad but shallow overview of many fundamental C++ concepts, so we can start writing some simple programs. Further chapters will explore those concepts in depth, or introduce new concepts.

Each chapter has a theme, with most of the lessons underneath it being generally related to that theme. There is no suggested amount of time that you should spend with each lesson or chapter; progress through the material at a pace that is comfortable for you.

Goals

Before we get started, let's cover a few important goals for these tutorials:

- Cover general programming topics as well as C++. Traditional textbooks do a pretty good job of teaching the basics of a given programming language, but they often do not cover other programming topics that are incidental to the language. For example, books will omit sections on programming style, common pitfalls, debugging, good/bad programming practices, and testing. Consequently, by the time you finish the book, you may understand how to program in a language, but you might also have picked up bad habits that will come back to bite you later! One of the goals of these tutorials is to make sure that all of these incidental topics are covered along the way, in the sections where it naturally makes sense to discuss them. When you finish, you will not only know how to program in C++, you will know how NOT to program in C++, which is arguably as important.
- Provide a lot of examples. Most people learn as much or more from following the examples as they do from reading the text. These tutorials will endeavor to provide plenty of clear, concise examples to show the application of concepts you are learning. We will also avoid (as much as possible) the twin evils: the *ellipsis* (also known as ...), where a necessary or non-obvious part of an example is omitted in the interest of space, and the *unexplained new concept*, where a new concept that is integral to the example is introduced without any mention of what it is or how it works. Both of these tend to lead to getting stuck.
- Provide practice programs. The end of many lessons and sections will contain some exercises that you can attempt to answer on your own, along with solutions. You can compare your solution against ours to see what we did differently, or, if you get stuck, how we solved the problem. Then you can go back and refocus on the areas you need more work on.
- Most importantly: have fun. Programming can be a lot of fun, and if you're not generally having fun, you're not in the right mindset to be programming. Tired or unhappy programmers make mistakes, and debugging code tends to take much longer than writing it correctly in the first place! Often you can save yourself some time by going to bed, getting a good night's sleep, and coming back to a problem in the morning.

Getting the most out of these tutorials

As you go through these tutorials, we recommend a number of practices to maximize your learning experience:

- Type in the examples *by hand*. This will help you learn where you commonly make errors, as well as becoming familiar with various diagnostic messages that result from typos. As you enter each line, think about what it does, and how it contributes to the overall program. If you encounter anything that you don't understand, that's something to investigate further.
- As you make mistakes or find bugs in your programs, fix them. Try to solve your own problems before asking others for help. Learning how to find and fix errors is a critical skill to successful programming. Don't neglect learning how to use a debugger (we'll explain how in a future chapter) – it's a key tool in figuring out where your programs are going wrong.
- Experiment with the examples. Change numbers and text to see what happens. Modify the programs to do additional things (e.g. if a program adds two numbers, make it add three numbers). Try to find different ways to break the programs (if a program asks you to enter a number, try entering a letter instead and see what happens). You'll learn more by modifying the examples than by simply following them.
- Plan to spend some time with the quizzes. If you're new to programming, you may find these challenging (and that's normal, as your brain acclimates to the programming mindset). Don't be discouraged if you don't get the right answer the first time. You may need to try several different approaches before you find a path to success. It's okay to look at the answer if you're really stuck. Just make sure you understand how the provided answer works before proceeding.
- Write your own short programs using the concepts you have learned. This will reinforce your learning and improve your retention.

Common site-related questions

Q: How do I sign up for the site? How do I get a login?

All parts of this site are accessible anonymously – therefore, no user account or signup is needed!

Q: Is there a PDF version of this site available for offline viewing?

Unfortunately, there is not. The site is able to stay free for everyone because we're ad-sponsored – that model simply doesn't work in PDF format. You are welcome to convert pages from this website into PDF (or any other) format for your own private use, so long as you do not distribute them.

Q: What should I do if I get stuck on a concept?

If you don't understand something or feel stuck:

- Read through the comments. Other readers may have encountered similar challenges.
- Scan through the next lesson or two in the series – your question may be answered there.
- Use a search engine to see if your question (or error message) has been addressed elsewhere.
- Ask an AI to explain something to you. Keep in mind that the information provided by AI may be fully or partially incorrect.
- Ask your question on a site that is designed for programming Q&A, like Stack Overflow.

If all else fails, skip the material you don't understand, and come back to it later. You may find that something that was hard to understand is easier with the additional knowledge and context provided by other articles.

Q: What do I do if I forget what something means?

Use our Site index to look up any topics you want to know more about there, and you'll find links to the lessons where that topic is discussed. The site index is also accessible from every lesson via a link in the title bar.

You might also try this glossary of C++ terms, provided by Bjarne Stroustrup (the creator of C++).

Q: Do you have any similar sites for other programming languages?

Nope. Maintaining this site requires a *lot* of time. Doing other languages would require being able to clone myself a few times.

Q: Can you do a dark mode for this site?

Not easily, but you can! See darkreader.org.

Finally, one small nag: This site is free because it is ad-supported. If you find yourself enjoying the lessons, please consider disabling your ad blocker.

Alright, let's get on with it!

What language standard is my compiler using?

The following program is designed to print the name of the language standard your compiler is currently using. You can copy/paste, compile, and run this program to validate that your compiler is using the language standard you expect.

PrintStandard.cpp:

```
        // This program prints the C++ language standard your compiler is currently using
        // Freely redistributable, courtesy of learncpp.com (https://www.learncpp.com/cpp-tutorial/what-language-
standard-is-my-compiler-using/)

        #include <iostream>

        const int numStandards = 7;
        // The C++26 stdCode is a placeholder since the exact code won't be determined until the standard is finali
        const long stdCode[numStandards] = { 199711L, 201103L, 201402L, 201703L, 202002L, 202302L, 202612L};
        const char* stdName[numStandards] = { "Pre-C++11", "C++11", "C++14", "C++17", "C++20", "C++23", "C++26" };

        long getCPPStandard()
        {
            // Visual Studio is non-conforming in support for __cplusplus (unless you set a specific compiler flag,
you probably haven't)
            // In Visual Studio 2015 or newer we can use _MSVC_LANG instead
            // See https://devblogs.microsoft.com/cppblog/msvc-now-correctly-reports-__cplusplus/
        #if defined (_MSVC_LANG)
            return _MSVC_LANG;
        #elif defined (_MSC_VER)
            // If we're using an older version of Visual Studio, bail out
            return -1;
        #else
            // __cplusplus is the intended way to query the language standard code (as defined by the language stan
            return __cplusplus;
        #endif
        }

        int main()
        {
            long standard = getCPPStandard();

            if (standard == -1)
            {
                std::cout << "Error: Unable to determine your language standard.  Sorry.\n";
                return 0;
            }

            for (int i = 0; i < numStandards; ++i)
            {
                // If the reported version is one of the finalized standard codes
                // then we know exactly what version the compiler is running
                if (standard == stdCode[i])
                {
                    std::cout << "Your compiler is using " << stdName[i]
                        << " (language standard code " << standard << "L)\n";
```

```
                break;
            }

            // If the reported version is between two finalized standard codes,
            // this must be a preview / experimental support for the next upcoming version.
            if (standard < stdCode[i])
            {
                std::cout << "Your compiler is using a preview/pre-release of " << stdName[i]
                    << " (language standard code " << standard << "L)\n";
                break;
            }
        }

        return 0;
    }
```

Build or runtime issues

If you get an error while trying to build this, you may have your project set up incorrectly. See 0.8 – A few common C++ problems for advice on some common issues. If that does not help, review the lessons starting from 0.6 – Installing an Integrated Development Environment (IDE).

If the program prints “Error: Unable to determine your language standard”, your compiler may be non-conforming. If you are using a popular compiler and this is the case, please leave a comment below with relevant information (e.g. the name and version of your compiler).

If this program prints a different language standard than you were expecting:

- Check your IDE settings to ensure your compiler is configured to use the language standard you expect. See 0.12 – Configuring your compiler: Choosing a language standard for more information on how to do this for some of the major compilers. Make sure there are no typos or formatting errors. Some compilers require setting the language standard for each project rather than globally, so if you’ve just created a new project, this may be the case.
- Your IDE or compiler may not even be reading the configuration file you’re editing (we see occasionally reader feedback on this with VS Code). If this seems like the case, please consult documentation for your IDE or compiler.

Q: If my compiler is using a preview/pre-release version, should I go back one version?

If you are just learning the language, it’s not necessary. Just be aware that some features from the upcoming version of the language may be missing, incomplete, buggy, or may change slightly.